Reward Engineering Playbook: A Systematic Study of Dense Reward Geometry for Robot Manipulation

Anonymous Author(s)
Affiliation
Address
email

Abstract: Dense reward design for robot manipulation is a persistent bottleneck: small changes in reward formulas can produce qualitatively different learning outcomes, yet the field lacks principled guidance for making these choices. We present a systematic empirical study of dense reward geometry using ManiSkill3's tabletop manipulation suite as a controlled testbed, injecting reward variants as environment parameters and holding algorithms and tasks fixed to isolate causal effects. Our experiments establish two core findings. First, a fundamental *robustness-ceiling tradeoff*: infinite-support rewards (e.g. `tanh`) trap policies in *hover attractors*—stable sub-optimal states where the shaped reward is high but task completion is not—while finite-support rewards (`concave_truncated`) resist this but create blind regions for hard initialisations. Within-episode annealing resolves the tradeoff on flat tasks (PickCube: 0.979, PushCube: 0.958, LiftPegUpright: 0.854) but catastrophically fails on hierarchical tasks (StackCube: mean peak 0.063 vs. 1.000 for finite-support)—a complete ranking inversion caused by the time clock decoupling from task progress at stage boundaries. Second, this failure motivates a *theoretically grounded* alternative: `adaptive_concave_distance`, a closed-form static reward in which the support radius self-adapts to the agent's current distance from the goal via $k(d) = k_{\min} + (k_{\max} - k_{\min})e^{-\alpha d}$. We derive conditions on $k_{\min}$, $k_{\max}$, α governing the reward's geometry, and show empirically that it matches `concave_truncated`'s peak performance on StackCube (mean peak 0.979 vs. 1.000) while converging approximately $2\times$ faster. The result is a *reward engineering playbook* with a theoretically principled base function for both flat and hierarchical tasks.

Keywords: reward shaping, dense rewards, robot manipulation, reinforcement learning, reward design, empirical study

1 Introduction

Reinforcement learning for robot manipulation requires dense reward functions to guide exploration: purely sparse rewards that fire only at success are rarely sufficient for contact-rich, multi-stage tasks. Yet practitioners designing these dense rewards have few principled guidelines [1]. The standard approach is to write a distance-based potential that decreases toward the goal, add binary stage bonuses, and tune scale factors empirically until training converges. This process is slow, non-transferable, and yields little understanding of which mathematical properties of the reward function matter.

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

Given that the task and algorithm are fixed, how does the geometry *of the dense reward function—its shape, curvature, support, smoothness, and staging—causally affect learning dynamics and final performance?*

This is distinct from several related lines of work. Automated reward generation methods [2] improve the speed of reward iteration but treat reward code as a black box to be searched rather than a structured object to be understood. Reward shaping theory [3, 4] provides guarantees about policy invariance under specific transformations but says little about optimization speed or empirical ranking among non-equivalent reward functions. What is missing is a controlled, systematic empirical study of reward geometry under on-policy learning in manipulation.

We address this gap by decomposing dense manipulation rewards into four independently parameterizable dimensions (§3), implementing them as environment kwargs in ManiSkill3 (§4), and sweeping across combinations while holding everything else fixed.

1.1 Contributions

C1. Reward geometry taxonomy. We decompose dense manipulation rewards into four independently controllable axes and implement them as environment kwargs across 11 ManiSkill3 tabletop tasks (§3, Appendix A).

C2. Hover attractor and robustness-ceiling tradeoff. We identify and characterise the *hover attractor*—a systematic failure mode of infinite-support rewards—and the fundamental tension between hover attractors and blind regions that no static reward geometry resolves (§6.1).

C3. Mechanistic account of hierarchical failure. We explain why within-episode time annealing, the best variant on all flat tasks, catastrophically fails on StackCube (mean peak 0.063): the episode clock decouples from task progress at stage boundaries—a complete ranking inversion (§6.4).

C4. REPCon: distance-adaptive reward theory and practice. We derive a first-principles theory of distance-adaptive reward geometry (§5), producing `adaptive_concave_distance` (REPCon unit)—a closed-form static reward matching the best hierarchical reward's peak performance (0.979) while converging $\approx 2\times$ faster.

2 Related Work

Reward shaping theory. Ng et al. [3] established that potential-based shaping does not alter the optimal policy in the limit, but this result does not address optimization dynamics under finite-sample on-policy methods. Devlin and Kudenko [4] extended the theory to dynamic potential functions. Our work is empirical and focuses on how shaping geometry affects convergence speed and stability rather than asymptotic correctness.

Automated reward design. EUREKA [2] uses LLMs to generate reward code and refine it via evolutionary search, achieving impressive results across locomotion and manipulation tasks. However, the method treats reward formulas as programs to be searched, not as geometric objects to be understood. Our work is complementary: we aim to understand *why* certain reward structures work, which could inform better search spaces for automated methods.

Hover attractors and reward support structure. Trott et al. [5] identified a failure mode in distance-based shaping where policies stabilise at a sub-optimal hovering distance rather than committing to contact (*"keeping your distance"*), and proposed sibling rivalry as a fix. Our work identifies the same phenomenon, names it the *hover attractor*, and proposes a cleaner geometric fix via distance-adaptive support. Nilaksh et al. [6] use barrier-function-inspired reward shaping to improve convergence speed by 1.4–2.8×; their approach is structurally related to REPCon in that

both modify the reward's distance-field geometry, but differ in derivation and the specific geometric properties controlled.

Dense reward design for manipulation. Wu et al. [7] learn dense rewards for contact-rich tasks (peg insertion, USB insertion) from self-supervised video signals, demonstrating that carefully shaped dense rewards are critical for hard contact tasks. Tang et al. [8] use SDF-based reward shaping for assembly sim-to-real transfer, showing that reward geometry matters for real robot performance. Both works motivate principled reward geometry for contact-rich manipulation; neither studies the functional-form tradeoffs that are the focus of our work.

Empirical studies of RL design choices. Andrychowicz et al. [9] conducted a large-scale study of on-policy RL design choices (architecture, normalization, etc.) but did not focus on reward geometry. To our knowledge, no comparable study exists for manipulation-specific reward function design.

Simulation infrastructure. ManiSkill3 [10] provides GPU-parallelized simulation, a standardized API across 18+ tabletop manipulation tasks, and built-in support for PPO baselines, making it ideal for controlled reward geometry sweeps.

Hierarchical task reward design. DrS [11] learns reusable dense reward functions for multi-stage tasks by distilling sparse success signals into per-stage dense rewards. Like our work, it studies stage-decomposed reward design; unlike ours, it focuses on reward *magnitude* learned from data, not the *functional form* (curvature, support) analyzed analytically. Hierarchical Potential-Based Reward Shaping (HPRS) [12] extends Ng et al.'s potential-based framework to task specifications, but again studies magnitude via potential functions rather than geometry. Self-Adaptive Reward Shaping [13] adapts reward weights online but does not study the geometric properties of the shaping function itself.

Hierarchical reinforcement learning. Option-Critic [14] and HIRO [15] learn temporal abstractions (options and sub-goals respectively) to decompose hierarchical tasks. Our approach is complementary: task stages are *given* by the task's `evaluate()` function, so we do not need to learn when to switch stages. We study how the reward geometry within each known stage affects learning. HER [16] uses goal-conditioned relabeling to handle sparse rewards; our work instead investigates how the dense shaping geometry interacts with stage structure.

Curriculum learning. Narvekar et al. [17] provide a framework for curriculum RL in which training instances are ordered by difficulty. Our within-episode annealing (`adaptive_tanh_episode`) is a special case of curriculum learning where the geometry of the reward function itself changes within an episode. The stage-conditioned extension (`adaptive_tanh_stage`) conditions the curriculum on task progress rather than wall-clock time. From Sparse to Dense [18] motivates sparse-to-dense reward transitions from biological learning; our adaptive annealing formalizes this intuition as a within-episode geometric schedule.

Concurrent work. Several 2026 preprints explore related but distinct directions: reward landscape analysis for model-based planning, adaptive reward weighting, and two-stage curriculum design. These address different algorithmic levers (planning objectives, weight scheduling, task ordering) rather than the intrinsic geometry of the dense shaping function itself.

3 Reward Geometry Taxonomy

We decompose dense manipulation rewards into four independently controllable dimensions. Each dimension is exposed as an environment kwarg (e.g., `reach_variant="concave_truncated"`), enabling factorial sweeps without modifying the algorithm or task.

3.1 Dimension 1: Reaching Shape

The reaching component $r_d : \mathbb{R}_{\geq 0} \to \mathbb{R}$ maps end-effector-to-object distance d to a scalar reward. We study eight static families and three adaptive variants that vary in curvature, support, and monotonicity (Table 1).

Table 1. Reaching reward shape variants. All satisfy $r_d(0) = 1$. $\nabla r_d|_{d=0}$ is the gradient at the goal. The bottom three rows are the adaptive variants central to this paper.

Name	Formula	Curvature	$\nabla r\|_{d=0}$
`tanh`	$1 - \tanh(5d)$	Convex	-5
`linear`	$\max(0, 1 - 5d)$	Linear	-5
`exponential`	e^{-5d}	Convex	-5
`quadratic`	$\frac{1}{1+25d^2}$	Concave peak	0
`tanh_sq`	$1 - \tanh(25d^2)$	Concave, no cutoff	0
`concave_truncated`	$\max(0,\ 1 - 25d^2)$	Concave, $r = 0$ for $d > 0.2$	0
`oscillatory`	$(1 - \tanh(5d)) + 0.05\sin(20d)$	Non-monotone	-5
`sparse_threshold`	$\mathbf{1}[d < 0.05]$	Discontinuous	—
`adaptive_tanh_episode`	$1 - \tanh(k(t) \cdot d)$, $k(t) = 2{+}18\frac{t}{T}$	Convex, time-varying	$-k(t)$
`adaptive_tanh_stage`	$1 - \tanh(k(t_s) \cdot d)$, k resets per stage	Convex, stage-varying	$-k(t_s)$
`adaptive_concave_distance`	$\max\big(0,\ 1 - (k(d) \cdot d)^2\big)$, $k(d) = k_{\min} + \Delta k\, e^{-\alpha d}$	Concave, k varies with d	0

3.2 Dimension 2: Stage Gating

Many manipulation tasks are hierarchical: the robot must reach, then grasp, then place. We study how the placement reward r_p is conditioned on the grasping phase variable $c_t \in \{0, 1\}$ (or a continuous proxy):

- `hard`: $r_p \cdot c_t$ (binary mask; baseline)
- `soft`: $r_p \cdot \sigma(k(\frac{1}{2} - \text{openness}_t))$ (continuous gripper closure)
- `additive`: r_p (no gate; all stages always active)
- `curriculum`: hard gate for first 40% of episode, then additive

3.3 Dimension 3: Terminal Bonus

At task success, we study three treatments of the terminal reward:

- `hard_jump`: override $r_t = r_{\max}$ at success (baseline)
- `smooth`: additive bonus $r_t \mathrel{+}= 0.5\, r_{\max} \cdot \mathbf{1}[\text{success}]$
- `none`: pure shaping; no terminal override

3.4 Dimension 4: Normalization

[TODO: Planned: unnormalized dense, normalized $[0, 1]$, z-score per batch, clipped. Currently using ManiSkill3's built-in normalized dense reward (divide by $r_{\max}$).]

4 Experimental Setup

Tasks. We study four tasks spanning the main structural archetypes in tabletop manipulation: PickCube-v1 (canonical pick-and-place, flat), PushCube-v1 (pushing without grasping, flat), LiftPegUpright-v1 (rotation-dominated, flat), and StackCube-v1 (two-stage hierarchical). PlaceSphere-v1 (three-stage hierarchical) is planned next. A full listing of the 11 tasks with reward variant support is in Appendix A.

Algorithm. All experiments use PPO [19] via the ManiSkill3 fast GPU baseline (256 parallel environments, 50-step rollouts). Flat tasks use 5M total timesteps; StackCube-v1 uses 40M to allow the harder hierarchical task sufficient training budget. We plan to extend to SAC [20] and a model-based method to assess algorithm-dependence of our findings.

Infrastructure. Reward variants are injected as environment kwargs, requiring no algorithm modification. This is implemented in a dedicated `reward_variants` module with three independent axes.

Evaluation. Each variant is evaluated with a *deterministic* policy (mean action, no noise) on 16 parallel environments every 25 training iterations. Primary metric: `eval/success_once`. Secondary metrics: eval return, explained variance, gradient norm, value loss, and steps to 80% success. Train/eval success gap is tracked as a reward hacking diagnostic.

Compute. RTX 4070 laptop (8GB VRAM). At 256 envs and 5M steps, one run takes approximately 45–60 minutes. Multi-seed validation (3 seeds per configuration) is feasible within the project timeline.

5 Theory of Reward Geometry

5.1 From Static to Adaptive k: The Design Progression

The experiments in Section 6 trace a natural arc in the design of reaching rewards. The concave truncated baseline has the form

$$r_d(d) = \max\big(0,\, 1 - (k\,d)^2\big), \tag{1}$$

with fixed k. This is a concave parabola that is exactly zero beyond $d = 1/k$: the *support radius* is $R = 1/k$. The core problem (F3) is that R must simultaneously be large enough to provide gradient signal from hard initialisations and small enough to prevent a soft hover attractor near the goal. No single value of k satisfies both.

The time-adaptive variants (`adaptive_tanh_episode`, `adaptive_tanh_stage`) address this by making k a function of elapsed time within an episode, starting small (wide R) and growing large (tight R). This resolves the tradeoff on flat tasks but fails on hierarchical ones because *time is a poor proxy for task progress*: by mid-episode on StackCube, the time clock has already tightened k while the robot is still in stage 1 (F10).

The natural resolution is to let k depend directly on the current distance d:

- **When far from the goal** (d large): use small k (wide R) to ensure the policy receives a gradient regardless of initialization.
- **When close to the goal** (d small): use large k (tight R) to create strong inward pull and suppress the soft hover attractor.

This gives a reward that is *geometrically self-adapting*: the support radius $R(d) = 1/k(d)$ automatically widens or narrows based on where the policy is, with no internal state or time clock.

5.2 The Distance-Adaptive Reward Function

We parametrize $k(d)$ with an exponential interpolation between two extreme values:

$$k(d) = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha d}, \tag{2}$$

so $k(0) = k_{\max}$ (tight near goal) and $k(d) \to k_{\min}$ as $d \to \infty$ (wide far from goal). The full reward is

$$\boxed{r(d) = \max\Big(0,\; 1 - \big(k(d)\cdot d\big)^2\Big), \quad k(d) = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha d}.} \tag{3}$$

We use $k_{\min} = 2$, $k_{\max} = 20$, $\alpha = 10$ in all experiments reported here; these are the first configuration we tried and further tuning may yield better results.

REPCon and REPTan units. We give a name to this primitive. A **REPCon unit** is an instance of Eq. (3) — a concave distance-adaptive reward characterised by the triple $(k_{\min}, k_{\max}, \alpha)$. The triple encodes the geometric requirements of a single subtask: $k_{\min}$ sets the outer support radius (how far the agent may start), $k_{\max}$ sets the inner basin tightness (the precision required at success), and α controls the transition length scale between the two regimes. Replacing the concave base $\max(0, 1 - (k(d)d)^2)$ with a tanh base $1 - \tanh(k(d) \cdot d)$ gives the analogous **REPTan unit**, which has infinite support (no hard cutoff) but otherwise the same $k(d)$ structure.

A hierarchical task with n stages can in principle be equipped with n REPCon (or REPTan) units, each tuned to the geometry of its subtask. The current `adaptive_concave_distance` variant is the degenerate case where all stages share a single REPCon unit — which already achieves strong results, suggesting a well-chosen unit generalises across subtasks. Per-stage parameter tuning is a natural extension (Section 7).

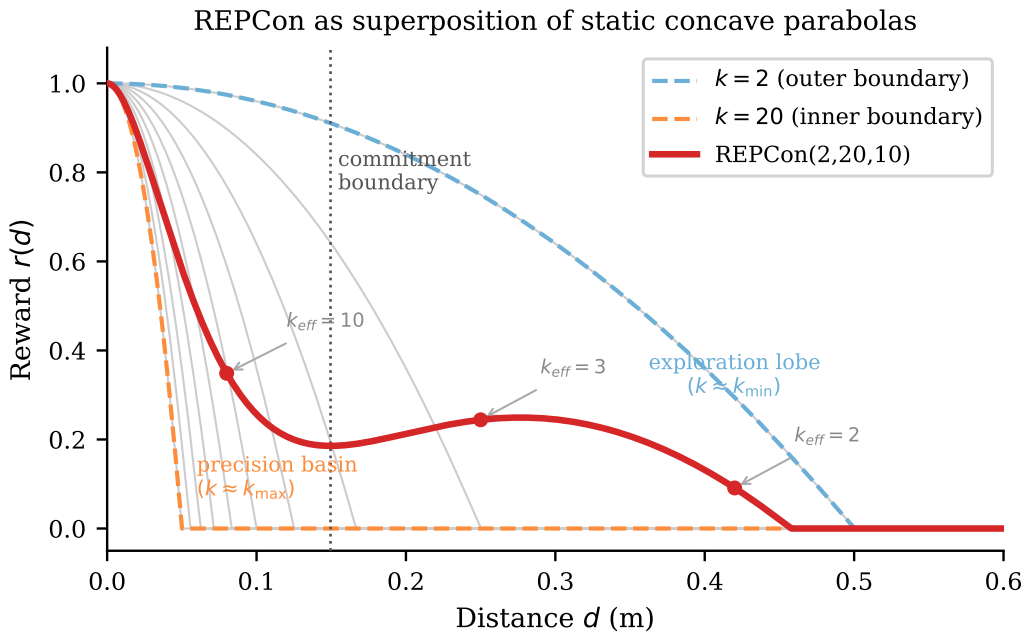

Figure 1. REPCon is a superposition of the family of static concave parabolas $\{\max(0, 1 - (kd)^2) : k \in [k_{\min}, k_{\max}]\}$. At distance d, the effective scale is $k(d)$ — wide ($k \approx k_{\min}$) when far, tight ($k \approx k_{\max}$) when close. The bold red curve traces through the family continuously. The dotted vertical line marks the commitment boundary at $d \approx 1.65/\alpha$.

5.3 Geometry of the Reward: The Scaled Distance $\varphi(d) = k(d) \cdot d$

Because $r(d) = \max(0, 1 - \varphi(d)^2)$, the shape of the reward is entirely determined by the *scaled distance* $\varphi(d)$. Understanding r reduces to understanding φ.

Outer cutoff. The reward reaches zero when $\varphi(d) = 1$. For large d, $k(d) \approx k_{\min}$, so the outer cutoff occurs near $d = 1/k_{\min}$. With $k_{\min} = 2$, this is ≈ 0.5m: numerically, $r(d) = 0$ for $d \gtrsim 0.46$m. This is the effective *outer support radius*.

Inner basin. Near $d = 0$, $k(d) \approx k_{\max} = 18$, so the reward gradient is steep — equal to the gradient of $\max(0, 1 - (18d)^2)$, a tight parabola with effective radius $1/k_{\max} \approx 0.056$m. The policy thus experiences a strong inward pull within the last ≈ 6cm of approach.

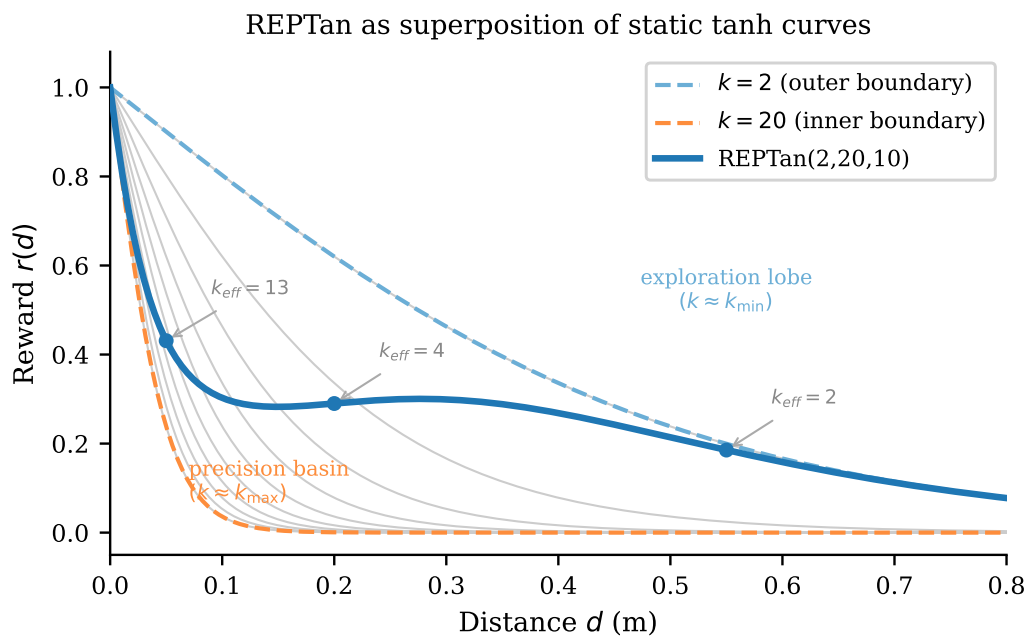

Figure 2. REPTan is a superposition of the family of static tanh curves $\{1 - \tanh(kd) : k \in [k_{\min}, k_{\max}]\}$. The bold blue curve threads through the family. Unlike REPCon, REPTan has infinite support (no hard cutoff) — it can resolve the hover attractor but retains a long, low-reward tail.

Does a moat exist? $\varphi'(d) = k(d) + k'(d)\, d = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha d}(1 - \alpha d)$. For $d > 1/\alpha$, the factor $(1 - \alpha d)$ is negative, so φ' can become negative (i.e., φ can have a local maximum). If φ has a local maximum φ^*, then:

- If $\varphi^* > 1$: the reward passes through zero twice, creating a genuinely *dead zone* (outer ring where $r = 0$) sandwiching an outer lobe of nonzero reward.
- If $\varphi^* < 1$: the reward never reaches zero at the local max — there is no dead zone, only a *shallow moat* (local minimum) in the reward profile, followed by a small outer bump.

The worst case of φ' occurs at $d^* = 2/\alpha$, where $\varphi'(d^*) = k_{\min} - (k_{\max} - k_{\min})e^{-2}$. Setting this to zero gives the moat-existence condition:

$$\frac{k_{\max}}{k_{\min}} > 1 + e^2 \approx 8.4. \tag{4}$$

With $k_{\min} = 2$, $k_{\max} = 20$: $k_{\max}/k_{\min} = 10 > 8.4$, so a moat *does* exist. Numerically, φ has a local max at $d \approx 0.165$m ($\varphi \approx 0.837$) and a local min at $d \approx 0.245$m ($\varphi \approx 0.828$), giving $r \approx 0.299$ at the moat and $r \approx 0.314$ at the outer bump. The moat is very shallow (depth ≈ 0.015) because $k_{\max}/k_{\min} = 10$ is only just above the threshold 8.4.

5.4 Parameter Roles

Each of the three parameters controls a distinct geometric property:

$k_{\min}$: outer support radius. $R_{\text{outer}} \approx 1/k_{\min}$. Decreasing $k_{\min}$ widens the region where the policy receives a nonzero gradient signal — improving robustness to hard initialisations. However, it also lowers the ratio $k_{\max}/k_{\min}$, which can eliminate the moat (Eq. (4)) and reduce the inner basin tightness. With $k_{\min} = 2$, $R_{\text{outer}} \approx 0.5$m.

$k_{\max}$: inner basin tightness and moat depth. $R_{\text{inner}} \approx 1/k_{\max}$ controls how tightly the gradient is concentrated near the goal. Increasing $k_{\max}$ sharpens the inner basin but also deepens the moat

(increases φ^*). From Eq. (4) and $\Delta k = k_{\max} - k_{\min}$, the moat vanishes when $\Delta k \leq k_{\min} e^2$, equivalently $k_{\max} \leq k_{\min}(1 + e^2)$. For $k_{\min} = 2$ this threshold is $\Delta k \approx 14.8$ (i.e. $k_{\max} \approx 16.8$): with $\Delta k \leq 14$ the reward is a smooth, strictly decreasing function; with $\Delta k \geq 15$ a moat appears. Our configuration ($k_{\max} = 20$, $\Delta k = 18$) sits just above the threshold, producing a very shallow moat. With $k_{\max} = 20$, $R_{\text{inner}} \approx 0.05$m.

α: commitment distance. α controls how quickly $k(d)$ transitions from $k_{\max}$ (near goal) to $k_{\min}$ (far away), and in doing so sets the distance at which the policy transitions from the *exploration regime* (wide support, shallow gradient) to the *precision regime* (tight basin, steep gradient). Intuitively, α answers the question: *at what distance from the goal does the policy begin committing to precision rather than gross approach?* Small α means the transition is gradual and spread over a large distance — the reward tightens slowly throughout the entire approach. Large α means the transition is sharp and localised: the policy experiences wide exploratory reward for most of the approach, then hits an abrupt shift into the precision basin close to the goal. The moat, when it exists, appears at $d \approx 1.65/\alpha$ (between $1/\alpha$ and $2/\alpha$), which is exactly the commitment boundary — the geometric point past which the policy is in the inner basin and should commit. With $\alpha = 10$, the commitment distance is $d \approx 0.165$m, well within the support radius (0.46m), and thus visible. Lowering α to 5 would push the commitment boundary toward $d \approx 0.35$m, where $\varphi > 1$ and the reward is already zero — the moat becomes invisible. Raising α to 15 moves the commitment boundary to $d \approx 0.11$m and deepens it to $r \approx 0.69$, making it a more substantial feature of the landscape.

We have not yet varied these parameters experimentally; the configuration $k_{\min} = 2$, $k_{\max} = 20$, $\alpha = 10$ is the first one we tested.

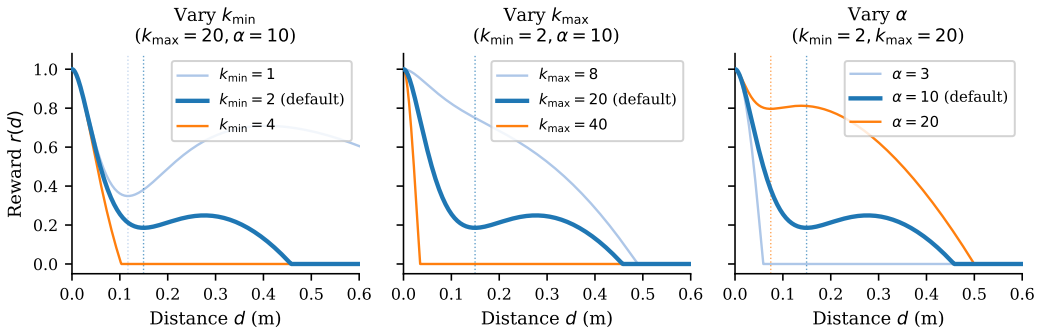

Figure 3. REPCon parameter sweep. **Left:** varying $k_{\min}$ changes the outer support radius $R_{\text{outer}} \approx 1/k_{\min}$. **Centre:** varying $k_{\max}$ changes inner basin tightness; at $k_{\max} \leq k_{\min}(1 + e^2) \approx 16.8$ the moat disappears. **Right:** varying α shifts the commitment boundary $d \approx 1.65/\alpha$ without changing the outer or inner extremes. Default configuration ($k_{\min} = 2, k_{\max} = 20, \alpha = 10$) shown bold red in each panel. Dotted verticals mark moat locations.

5.5 Why Distance-Adaptivity Resolves the Hierarchical Failure

The failure of time-based annealing on StackCube (F10) stems from the episode clock becoming decoupled from task progress at stage boundaries. Distance-based adaptivity eliminates this coupling entirely: $k(d)$ is a *pure function of the current robot–object distance*, with no memory of time, stages, or episode history.

When a stage transition occurs in StackCube (grasp achieved; now reach for placement target), d jumps from near-zero (successful grasp) to a large value (distance to placement location). This jump in d causes $k(d)$ to drop back toward $k_{\min}$, automatically widening the support radius for the new sub-goal — exactly the behavior that the stage-clock reset in `adaptive_tanh_stage` tried to engineer explicitly. Here it happens for free, as a consequence of the geometry.

More generally, `adaptive_concave_distance` is *geometrically adaptive but temporally static*: it is a closed-form function of d alone, with no internal state. The "adaptivity" lives in the reward

landscape's curvature, not in any schedule. Table 2 places this in the context of the full design progression.

Table 2. Arc of reward design decisions. Each row shows the mechanism and the failure mode that motivates the next row. REPCon eliminates the failure mode of all predecessors.

Design	Mechanism	Failure mode
Static infinite (`tanh`)	Fixed wide R	Soft hover attractor
Static finite (`concave_truncated`)	Fixed narrow $R = 1/k$	Hard attractor / blind region
Time-adaptive (`adaptive_tanh_episode`)	$k(t)$: small→large over episode	$t \neq$ progress on hierarchical tasks
Stage-adaptive (`adaptive_tanh_stage`)	$k(t_s)$, reset per stage	Stage clock still decoupled from d
Distance-adaptive (`adaptive_concave_distance`)	$k(d)$: large→small with d	Requires parameter tuning

5.6 Why REPCon Produces Consistent Onset: A Gradient Analysis

The key mechanism behind REPCon's fast, seed-robust onset is that its gradient is *nonzero across the entire initialization distribution from the first training step*, while being maximally concentrated near the goal at convergence. We make this precise.

Gradient of REPCon. For $r(d) = \max(0, 1 - \varphi(d)^2)$ where $\varphi(d) = k(d) \cdot d$, the gradient is:

$$\frac{dr}{dd} = -2\varphi(d) \cdot \varphi'(d) \quad \text{when } \varphi(d) < 1, \quad \text{else } 0, \tag{5}$$

where

$$\varphi'(d) = k(d) + k'(d) \cdot d = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha d}(1 - \alpha d). \tag{6}$$

Why onset is fast. At large d (exploration regime): $k(d) \to k_{\min}$, so $|\nabla r| \approx 2k_{\min}^2 d$ — linear in d, always positive up to the outer cutoff $R_{\text{outer}} \approx 1/k_{\min}$. A random policy at $d = 0.3$m receives gradient signal $\approx 2 \cdot (2)^2 \cdot 0.3 = 2.4$ — sufficient to bootstrap learning. The same policy under `concave_truncated`($k = 5$) at $d = 0.3$m: $r = 0$, gradient $= 0$ — a blind region with no signal. Since $R_{\text{outer}} = 0.5$m exceeds the maximum initialization distance (≈ 0.28m for PickCube), *every seed* starts inside the reward support, eliminating blind-region failures.

Why convergence is tight. Near $d = 0$ (precision regime): $k(d) \to k_{\max}$, so $|\nabla r| \approx 2k_{\max}^2 d$ — the same linear form but with $k_{\max}/k_{\min} = 10\times$ larger coefficient. At $d = 0.03$m: REPCon gradient $\approx 2 \cdot (20)^2 \cdot 0.03 = 24$, which is $10\times$ larger than the gradient at $d = 0.3$m in the exploration regime. The distance-adaptive $k(d)$ thus creates a *gradient amplification* near the goal: the same policy step size produces $10\times$ larger reward improvement when close vs. far. In contrast, $1 - \tanh(5d)$ at $d = 0.03$m gives gradient ≈ 4.85 — within $2\times$ of its value at large d, so there is no comparable concentration.

Signal ratio and the unique triad. Define the signal ratio $\rho = |\nabla r|_{d\to 0}/|\nabla r|_{d=R_{\text{outer}}}$. For REPCon: $\rho = (k_{\max}/k_{\min})^2 = 100$ — gradient is $100\times$ stronger near the goal than at the outer boundary. No static k achieves all three simultaneously: (1) zero beyond R_{outer} (no hover attractor), (2) strong gradient near goal (fast convergence), and (3) nonzero gradient at initialization distances (no blind region). Distance-adaptive $k(d)$ resolves the triad by decoupling the three requirements: $k_{\min}$ governs the outer support, $k_{\max}$ governs the inner basin, and α controls the transition between them.

The REPCon Triad

REPCon is the unique natural family achieving all three simultaneously: **(1)** compact support, eliminating hover attractors; **(2)** gradient amplification near the goal, enabling fast convergence; **(3)** wide outer lobe covering all initialization distances, ensuring no blind-region failures. No static k achieves all three; distance-adaptive $k(d)$ resolves the triad by decoupling the three requirements.

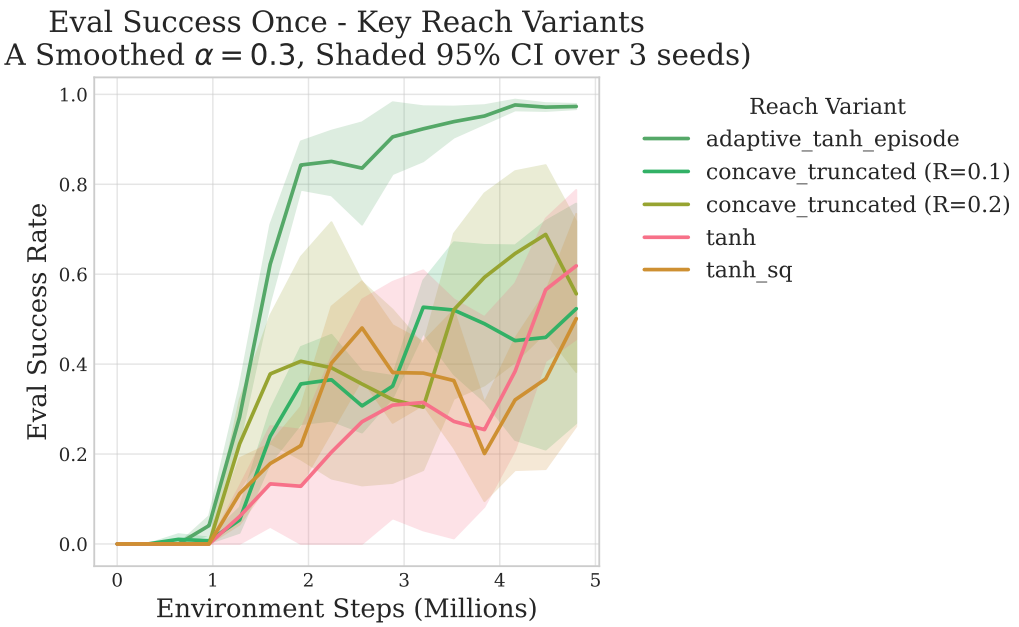

Figure 4. Key reach variants on PickCube-v1. Mean ± 95% CI over seeds $\{1, 42, 123\}$. `adaptive_tanh_episode` (green, bold) is the only variant that achieves near-perfect success consistently across all seeds.

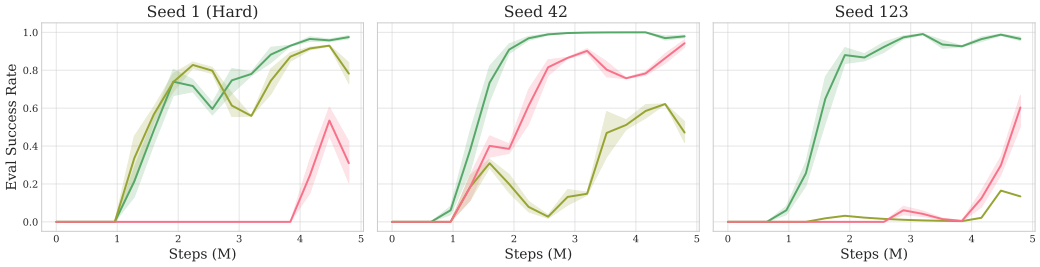

Figure 5. Per-seed learning curves for three representative variants. Seed 1 is a hard initialisation on which `tanh` and `concave_truncated` both struggle; `adaptive_tanh_episode` reaches 1.0 on all three seeds.

6 Results

All experiments use `gate=hard`, `terminal=hard_jump`, 256 parallel environments, seeds $\{1, 42, 123\}$.

6.1 PickCube-v1: Reach Shape Ablation

5M steps. Learning curves and 95% confidence intervals are shown in Figures 4–6. Table 3 reports final success rates.

F1 Hover attractors cause initialisation-dependent failure. `tanh` and `tanh_sq` reach 1.0 on easy seeds (42, 123) but completely fail on seed 1 (Figure 5, left panel). Formally, let $d(s)$ be the distance to the goal and $r_d(d)$ be the dense reward. A state s_h with $d_h = d(s_h) > 0$ acts as a *hover attractor* if the expected gradient of the advantage function $A(s, a)$ pushes the policy toward maintaining d_h rather than progressing toward $d = 0$. For infinite-support rewards like $r_d(d) = 1 - \tanh(kd)$, the gradient $\nabla_d r_d(d) = -k\text{sech}^2(kd)$ exists and is non-zero everywhere. This creates a continuous gradient ascent path toward a local region where the shaping reward is high, but crossing into the

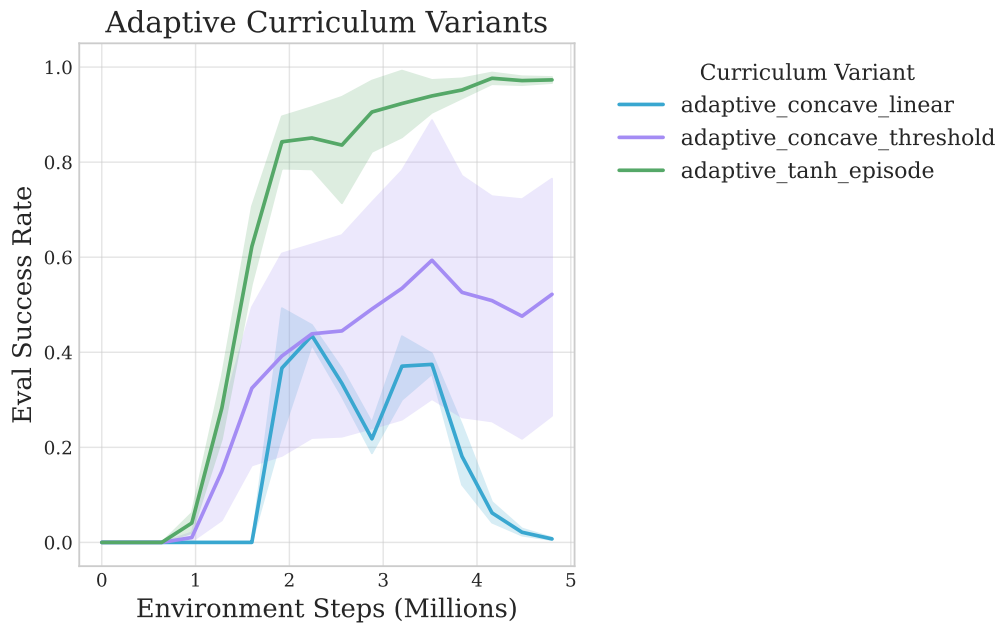

Figure 6. Adaptive curriculum variants versus static baselines. The linear schedule (`adaptive_concave_linear`) collapses because it shrinks the support radius continuously regardless of learning progress. The threshold schedule (`adaptive_concave_threshold`) fails on seed 1 due to a chicken-and-egg: it requires success to narrow the radius, but needs the narrow radius to achieve success. Within-episode annealing (`adaptive_tanh_episode`) breaks the deadlock by delivering the full wide-to-narrow curriculum in every episode from the start of training.

Table 3. Final `eval/success_once` at 5M steps (PickCube-v1). Seeds $\{1, 42, 123\}$ where available.

Variant	Seed 1	Seed 42	Seed 123	Mean $\pm$ Std
`tanh`	0.000	1.000	0.875	0.625 ± 0.445
`tanh_sq`	0.000	1.000	0.938	0.646 ± 0.457
`concave_truncated` (R=0.2)	0.625	0.312	—	0.469 ± 0.156
`concave_truncated` (R=0.1)	0.875	0.000	0.875	0.583 ± 0.412
`adaptive_concave_linear`	—	0.000	—	0.000
`adaptive_concave_threshold`	0.000	1.000	0.750	0.583 ± 0.425
`adaptive_tanh_episode`	**1.000**	**1.000**	**0.938**	$\mathbf{0.979 \pm 0.029}$

next task stage (e.g., grasping the object) requires a coordinated sequence of actions that temporarily lowers expected return. Thus, hard initialisations converge to this hover attractor ($d \approx 0.3$m) and never escape. Explained variance remains ≈ 0.96 throughout, confirming the failure is policy-level, not a value function issue [5].

F2 Finite support prevents hover attractors but causes mode-switching instability. `concave_truncated` (R=0.2) survives seed 1 (peak 0.9375) but oscillates dramatically and ends at 0.625 after 5M steps. Mathematically, the abrupt zero at $R = 0.2$m in $r_d(d) = \max(0, 1 - (d/R)^2)$ creates a non-differentiable boundary in the state-reward mapping. When the policy π_θ is near this boundary, slight errors in the value function $V(s)$ cause the advantage estimate to swing wildly between positive and negative depending on whether a sampled trajectory crosses into the $d < R$ zone. This discontinuity in the incentive boundary forces the policy to repeatedly cross it in both directions, producing *mode-switching* (alternating between exploring outside the zone and exploiting inside it) rather than stable convergence. Narrowing to R=0.1 slows initial learning $\approx 2.5\times$ and produces the same instability.

F3 The robustness-ceiling tradeoff is fundamental. Wide support (large R): initialisation-robust, hover-prone. Narrow support (small R): hover-resistant, mode-switching unstable. No static geometry resolves both simultaneously. This tradeoff is not captured by potential-based shaping theory [3], which addresses asymptotic optimality but not training dynamics under finite samples.

F4 Near-goal curvature does not independently predict performance. `tanh` (convex, $\nabla r|_{d=0} \neq 0$) and `tanh_sq` (concave, $\nabla r|_{d=0} = 0$) perform nearly identically across all seeds. H2 (stage-handoff geometry) is not confirmed as an independent effect; curvature matters only in conjunction with support structure.

F5 Progress-gated support narrowing fails on hard seeds (chicken-and-egg). `adaptive_concave_threshold` ($R : 0.4 \rightarrow 0.1$, triggered when `success` ≥ 0.5) achieves 1.0 on seed 42 and 0.75 on seed 123, but 0.0 on seed 1: the wide initial support R=0.4 creates an even stronger hover attractor than R=0.2, so the threshold is never crossed and R never narrows.

F6 Within-episode annealing resolves the tradeoff. `adaptive_tanh_episode` anneals the scale k from 2 to 20 over the max episode length T. Formally, at step $t \in [0, T]$:

$$r(d, t) = 1 - \tanh(k(t) \cdot d), \quad k(t) = 2 + 18 \cdot \frac{t}{T} \tag{7}$$

Early in each episode ($t \approx 0$), $k(t)$ is small, resulting in a wide gradient footprint that bootstraps exploration regardless of overall training progress. Late in each episode ($t \approx T$), $k(t)$ is large, concentrating the gradient strictly near the goal (the hover reward at $d = 0.2$m shrinks from $1 - \tanh(0.4) \approx 0.61$ to $1 - \tanh(4.0) \approx 0.004$). Because the curriculum is delivered within every episode, hard seeds get the wide bootstrap phase on every reset—breaking the chicken-and-egg. Results: mean 0.979 ± 0.029 versus next-best 0.646 ± 0.457 for `tanh_sq`.

6.2 Cross-Task Validation: PushCube-v1

PushCube-v1 requires the robot to push a cube to a target position without grasping—a flat task with no grasping stage. This tests whether the hover-attractor mechanism is specific to grasp-dependent tasks.

F7 Hover attractor failure generalises to non-grasping tasks. `adaptive_tanh_episode` outperforms `tanh` on PushCube (0.958 vs. 0.854), confirming that the hover-attractor mechanism is not specific to the grasping stage but affects any manipulation task requiring precise end-effector positioning.

6.3 Cross-Task Validation: LiftPegUpright-v1

LiftPegUpright-v1 requires the robot to grasp a horizontal peg and reorient it to vertical—a task dominated by a *rotation* reward component (cosine similarity of peg orientation), with reaching contributing only $\frac{1}{5}$ of the shaping signal.

Table 4. Final `eval/success_once` at 5M steps on LiftPegUpright-v1.

Variant	Seed 1	Seed 42	Seed 123	Mean ± Std
`tanh`	0.750	0.688	0.938	0.792 ± 0.107
`adaptive_tanh_episode`	**0.750**	**1.000**	**0.812**	$\mathbf{0.854 \pm 0.108}$

F8 Hover attractor severity scales with reaching reward dominance. On PickCube, reaching is the primary shaping signal and the hover attractor completely blocks seed 1 (0.0 for `tanh`). On LiftPegUpright, the rotation reward dominates (4× weight of reaching), and seed 1 is no longer catastrophically stuck: both variants achieve 0.75 on seed 1. This confirms that the hover attractor is not a fixed property of the reaching function alone, but depends on its *relative weight* in the total reward signal. When rotation or placement rewards co-dominate, the reaching shaping function has less capacity to create a stable attractor.

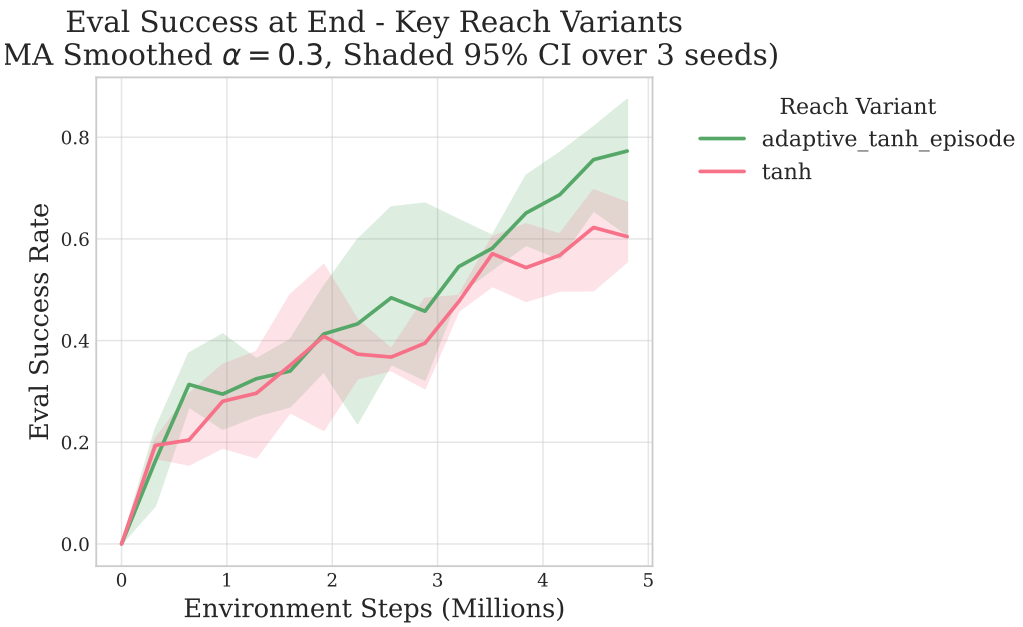

Figure 7. PushCube-v1: `adaptive_tanh_episode` maintains high performance (0.958 ± 0.029) compared to standard `tanh` (0.854 ± 0.128), demonstrating the robustness of within-episode annealing on a non-grasping task.

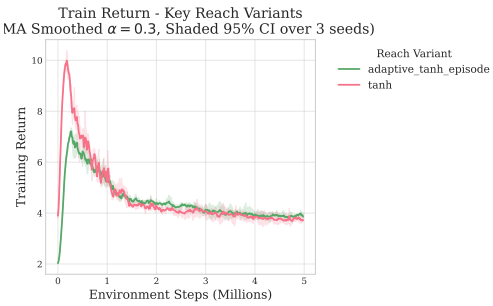

Figure 8. PushCube Training Returns

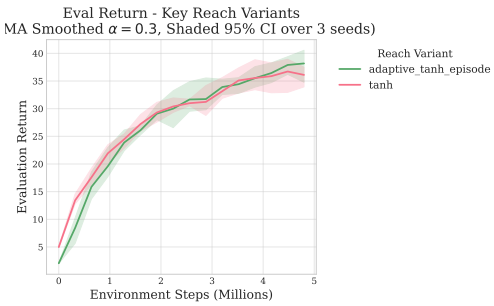

Figure 9. PushCube Eval Returns

F9 Within-episode annealing provides consistent gains across task types. Despite the diluted reaching signal, `adaptive_tanh_episode` still outperforms `tanh` in mean success (0.854 vs 0.792) and reduces inter-seed variance. Notably, `eval/success_at_end`—a harder metric requiring the peg to remain upright at episode end—improves from 0.104 (`tanh`) to 0.167 (`adaptive_tanh_episode`), a $\approx 60\%$ relative gain, suggesting that within-episode annealing also helps commitment at task completion regardless of task structure.

6.4 Cross-Task Validation: StackCube-v1

StackCube-v1 is a two-stage hierarchical task: the robot must grasp a cube, lift it, and place it precisely on top of a second cube. Crucially, success requires sustained, coordinated control across *two* contact stages with tight spatial tolerances. Runs use 40M steps.

F10 Within-episode annealing catastrophically fails on hierarchical tasks. `adaptive_tanh_episode` achieves mean 0.021 on StackCube—virtually zero—despite being the best variant on all three flat tasks. This is a complete ranking inversion.

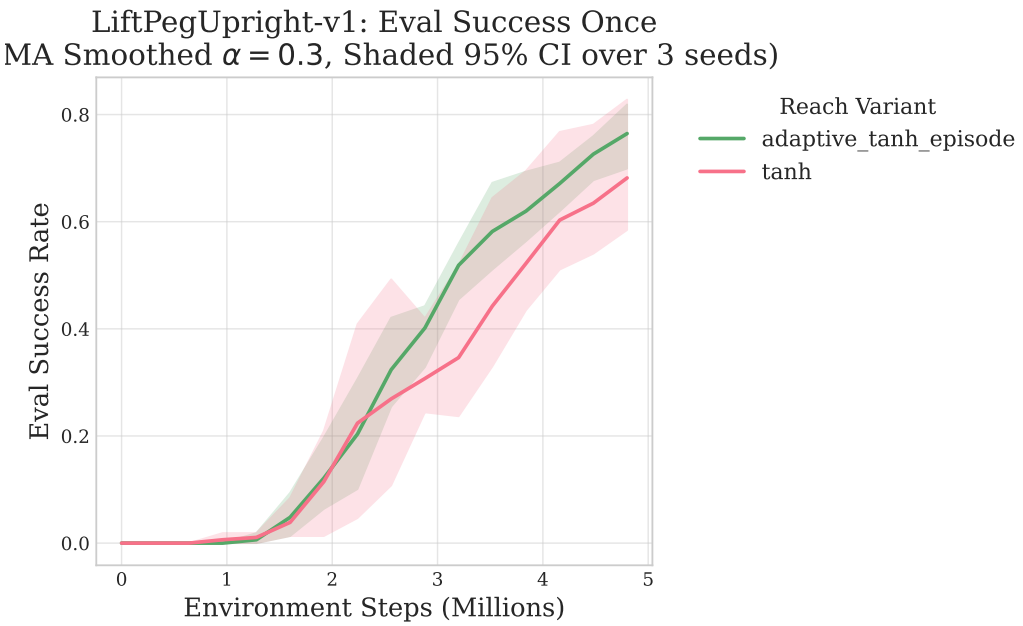

Figure 10. LiftPegUpright-v1: `eval/success_once` mean $\pm$ 95% CI over seeds $\{1, 42, 123\}$. Both variants reach similar final success rates, but `adaptive_tanh_episode` converges more reliably and shows lower inter-seed variance.

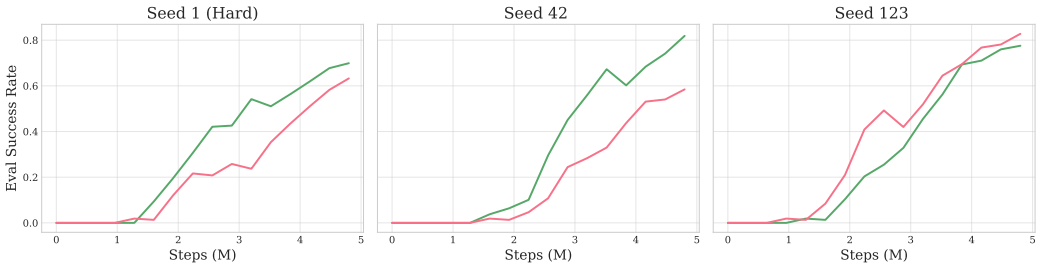

Figure 11. LiftPegUpright-v1 per-seed curves. Unlike PickCube, seed 1 is no longer a hard seed for either variant, suggesting that the hover attractor mechanism is diluted when reaching is not the dominant reward component.

The mechanism is the interaction between within-episode annealing and hierarchical task structure. In PickCube, the task is effectively *flat*: the robot reaches, grasps, and places in one continuous motion, so wall-clock episode time $\approx$ task progress. In StackCube, the task has *two committed contact stages*: grasp the lower cube, then place it on the upper cube. The within-episode annealing schedules k as a function of *time*, not *task progress*. By the mid-episode point when k is already large, the robot may still be in stage 1 (reaching for the lower cube), and the now-tight reaching gradient provides insufficient signal for a robot even slightly off-center. The schedule mismatch prevents the policy from ever consolidating a reliable grasp, collapsing performance.

F11 Finite support enables hierarchical manipulation; time-based annealing does not. `concave_truncated` achieves peak success 1.000 on all three seeds (Table 5), confirming that finite support is sufficient for hierarchical tasks. The zero-gradient zone at $d < R$ may actively *help*: once the robot is within reach, the reaching reward goes silent and stops competing with the downstream grasping and placement signal, allowing the policy to commit to contact. This is the inverse of the hover-attractor failure: reward *silence* near the goal creates a stable commitment region.

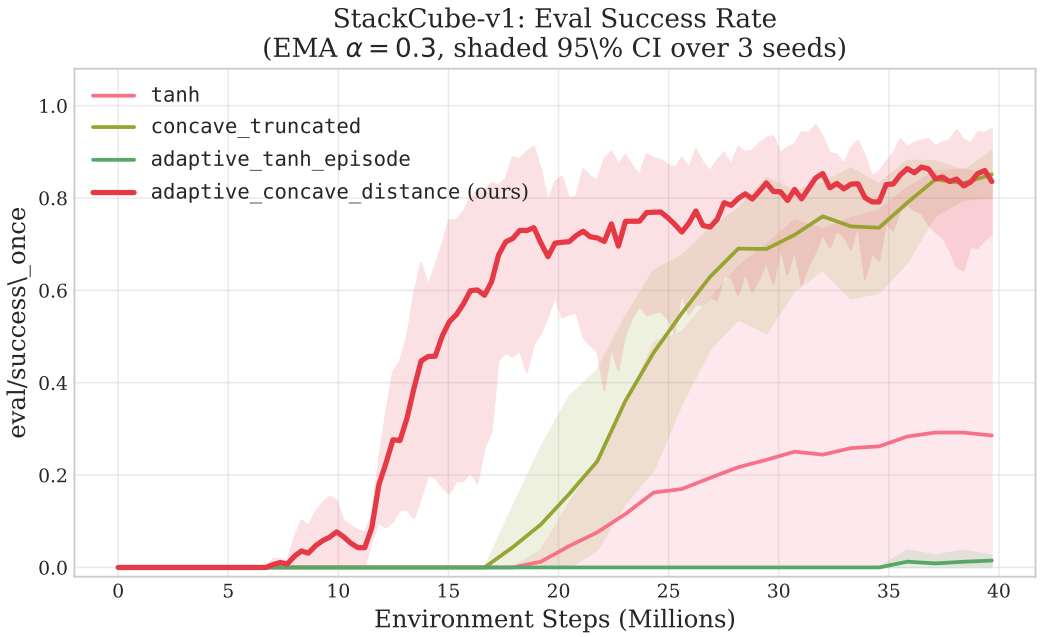

Figure 12. StackCube-v1: `eval/success_once` mean ± 95% CI over seeds $\{1, 42, 123\}$ (EMA smoothed α=0.3). `adaptive_concave_distance` and `concave_truncated` both reach high success; `adaptive_tanh_episode`—best on all flat tasks—collapses completely. `tanh` is seed-dependent (only seed 42 learns).

Figure 13. StackCube-v1 per-seed curves. `adaptive_concave_distance` reaches nonzero success on all three seeds by ∼7–12M steps, roughly 2× faster than `concave_truncated` (∼18–22M steps). `adaptive_tanh_episode` is at or near zero throughout on all seeds.

However, `concave_truncated` is slow to learn: the first nonzero success does not appear until ≈18–22M steps across seeds.

F12 Distance-adaptive reward matches peak performance with ≈ 2× faster convergence. `adaptive_concave_distance` achieves peak `eval/success_once` of 1.000 / 0.938 / 1.000 on seeds 1, 42, 123 (mean peak **0.979**), comparable to `concave_truncated` (mean peak 1.000), but with all three seeds showing nonzero success by ∼7–12M steps versus ∼18–22M steps for `concave_truncated` — roughly 2× faster onset. The final values (1.000 / 0.750 / 0.938) reflect late-training instability on seed 42, not a failure to learn: seed 42 peaks at 0.938 at ∼20M steps before partially decaying.

The mechanism follows directly from Section 5. Far from the goal ($d \approx 0.3$–0.5m), $k(d) \approx k_{\min} = 2$ and the outer support radius $R \approx 0.5$m provides gradient signal even from hard initialisations. Close to the goal ($d < 0.05$m), $k(d) \approx k_{\max} = 20$ tightens the basin, suppressing the soft hover attractor. Neither regime is hardcoded to a time or stage: when a stage transition occurs and d jumps to a large value, the reward automatically widens back to the exploration regime — without any clock reset.

Table 5. StackCube-v1 at 40M steps: peak and final `eval/success_once`. Peak = max over training; Final = last checkpoint. Best mean peak in bold.

	Peak success			Final success		
Variant	**S1**	**S42**	**S123**	**S1**	**S42**	**S123**
`tanh`	0.063	0.938	0.063	0.000	0.812	0.000
`concave_truncated`	1.000	1.000	1.000	0.875	0.812	1.000
`adaptive_tanh_episode`	0.063	0.000	0.125	0.063	0.000	0.000
`adaptive_concave_distance`	**1.000**	**0.938**	**1.000**	1.000	0.750	0.938

Mean peak: `concave_trunc.` 1.000; `adaptive_concave_dist.` **0.979**; `tanh` 0.354; `adaptive_tanh_ep.` 0.063

6.5 Summary: Task-Structure-Dependent Playbook

Table 6. Cross-task summary. Mean peak `eval/success_once` over seeds $\{1, 42, 123\}$. Best per task in bold. '—' = not yet evaluated. No single static reward achieves high performance on both flat and hierarchical tasks; REPCon (`adapt. conc. dist.`) is the first to approach this.

Task	**Type**	`tanh`	`concave_trunc.`	`adapt. tanh ep.`	`adapt. conc. dist.`
PickCube-v1	Flat	0.646	0.583	**0.979**	—
PushCube-v1	Flat	0.854	—	**0.958**	—
LiftPegUpright-v1	Flat	0.792	—	**0.854**	—
StackCube-v1	Hierarchical	0.354	1.000	0.063	**0.979**

Takeaway. *On flat tasks,* `adaptive_tanh_episode` *dominates by resolving the robustness-ceiling tradeoff within every episode. On hierarchical tasks, it catastrophically fails because the episode clock decouples from task progress at stage boundaries.* `adaptive_concave_distance` *(REPCon) is the single reward that works across both task types—stateless, closed-form, and geometry-grounded.*

Two patterns emerge. First, on flat tasks, within-episode annealing (`adaptive_tanh_episode`) consistently dominates: it resolves the robustness-ceiling tradeoff by delivering a wide-to-tight curriculum within every episode, ensuring hard seeds always receive the wide bootstrap phase. Second, on the hierarchical StackCube task, time-based annealing catastrophically fails (mean peak 0.063) because the episode clock decouples from task progress at stage boundaries. Finite-support `concave_truncated` succeeds (mean peak 1.000) via reward silence near the goal, but converges slowly ($\sim$18–22M steps to first success).

This discrete, task-type-dependent observation raises a deeper question: is there a single reward function that (a) resolves the hover attractor, (b) provides wide support from hard initialisations, and (c) does not depend on any time or stage clock? The theory in Section 5 and the results in F12 show that `adaptive_concave_distance` is a positive answer: by coupling the support radius directly to the current distance d, it eliminates both failure modes without any internal state. On StackCube it achieves mean peak 0.979 while converging $\approx 2\times$ faster than `concave_truncated`. Cross-task validation on flat tasks is the immediate next experiment (Section 7).

7 Planned Experiments

7.1 Cross-Task Validation of `adaptive_concave_distance`

The most immediate open question is whether `adaptive_concave_distance` maintains the strong performance of `adaptive_tanh_episode` on flat tasks. On a single-stage task, the distance d decreases monotonically throughout the episode, so the wide-to-tight transition of $k(d)$ mirrors the

wide-to-tight annealing of `adaptive_tanh_episode` — but gated by actual proximity rather than time. We predict no regression on PickCube, PushCube, or LiftPegUpright.

Success criterion: `adaptive_concave_distance` achieves mean peak ≥ 0.93 on PickCube-v1 (matching `adaptive_tanh_episode`'s 0.979 within tolerance) without any catastrophic failure seeds, and ≥ 0.93 on PushCube-v1. If confirmed, it becomes a *universal* base reward: no task-type selection required.

7.2 Parameter Sensitivity ($k_{\min}$, $k_{\max}$, α)

The current configuration ($k_{\min} = 2$, $k_{\max} = 20$, $\alpha = 10$) is the first we tested. Section 5 derives the moat-existence condition $k_{\max}/k_{\min} > 1 + e^2 \approx 8.4$ and characterises the role of each parameter. Key ablations:

- Vary $k_{\max} \in \{14, 16, 18, 22\}$ to move across the moat threshold and assess whether the moat is beneficial, neutral, or harmful.
- Vary $k_{\min} \in \{1, 2, 3\}$ to change the outer support radius $R_{\text{outer}} = 1/k_{\min} \in \{1.0, 0.5, 0.33\}$m.
- Vary $\alpha \in \{5, 10, 15\}$ to shift the moat location and transition sharpness.

These ablations will reveal which aspects of the geometry drive performance and provide guidance for applying the reward to new tasks with different length scales.

7.3 Deeper Hierarchies: PlaceSphere-v1

PlaceSphere-v1 is a three-stage task (reach, grasp, place) with 13 reward components. It is a stronger test of distance-adaptive rewards on hierarchical tasks because stage transitions cause d to jump *twice*, each time returning $k(d)$ to the wide-support regime. We plan 3 seeds comparing `tanh`, `concave_truncated`, and `adaptive_concave_distance`.

7.4 Gating and Terminal Dimensions

Fix `adaptive_concave_distance` as the reaching variant and sweep the gating and terminal bonus dimensions on PickCube-v1. Cross-validate top configurations on StackCube-v1 and PlaceSphere-v1.

7.5 Algorithm Generalization

Repeat the top configurations with SAC and a model-based method to assess whether the geometric findings are specific to on-policy PPO or more general.

7.6 Sim-to-Real (Stretch)

Validate the top configuration on the SO-101 robotic arm via the `lerobot-sim2real` bridge supported in ManiSkill3.

8 Discussion

From task-type selection to a universal geometry. The initial playbook finding—flat tasks favour time-based annealing, hierarchical tasks favour finite support—is real but incomplete. It describes a consequence of a deeper principle: reward support should be wide when the agent is far from the goal and tight when close. Time is a good proxy for distance on flat tasks (where d decreases monotonically during a well-behaved episode) but a poor proxy on hierarchical tasks (where d resets at each stage). `adaptive_concave_distance` removes the proxy entirely by coupling the support radius directly to d, yielding a reward that is self-adapting, stateless, and task-structure-agnostic in principle.

The reward geometry perspective. Existing reward design literature focuses on magnitude (reward weights, normalisation) or potential-based structure (shaping theorems). The present work argues that *geometry*—the shape of the reward as a function of distance—is an independently important design axis. Related geometric perspectives include barrier-function-inspired shaping [6] and SDF-based rewards for sim-to-real transfer [8]; our contribution is a systematic framework characterising these choices analytically. The hover attractor, the mode-switching instability, the schedule-mismatch failure, and the faster convergence of `adaptive_concave_distance` are all consequences of reward geometry rather than magnitude. Potential-based theory [3] is silent on all of them.

Limitations and open questions. The parameter triple $(k_{\min}, k_{\max}, \alpha)$ introduces new hyperparameters. We have only tested one configuration; Section 7 describes a systematic ablation. The moat condition $k_{\max}/k_{\min} > 1 + e^2$ is a precise analytical criterion but whether the moat is beneficial, neutral, or harmful in practice is an open empirical question. Finally, our experiments use PPO exclusively; off-policy methods may interact differently with reward geometry because they do not replay the same near-goal trajectories within a single update batch.

REP units as a reward design language for long-horizon tasks. The REPCon and REPTan units introduced in Section 5 suggest a broader design language: a long-horizon task with n stages can be equipped with n REP units, one per stage, each with its own $(k_{\min}, k_{\max}, \alpha)$ triple encoding that stage's geometric requirements. $k_{\min} \approx 1/d_{\max}$ sets the outer support to match the maximum starting distance for that stage; $k_{\max} \approx 1/\epsilon$ encodes the required precision at success; and α controls how sharply the reward transitions between the two regimes. The current work uses a single shared REPCon unit across all stages — the degenerate case — and already achieves strong results, suggesting the geometry generalises well. Per-stage tuning is a natural next step, particularly for tasks with heterogeneous subtask geometries (e.g. free-space reaching followed by precision insertion).

A further extension is to *infer* REP unit parameters from video demonstrations rather than specifying them manually. A demonstration provides a distance profile $d(t)$ for each subtask: the maximum distance reveals $k_{\min}$, the success tolerance reveals $k_{\max}$, and the sharpness of the $d(t)$ drop reveals α. Stage boundaries are identified as points where $d(t)$ jumps as the next subgoal activates. This would make the entire dense reward function inferrable from a small number of demonstrations with no reward engineering — connecting the REP unit framework to imitation learning and inverse reward design.

REP units on learned task-progress signals. Throughout this work d denotes Euclidean end-effector distance, which is a good proxy for inverse task progress on reaching tasks (where $d \to 0$ corresponds to task completion) but a poor one for tasks with strong orientation or contact requirements, such as peg insertion. A natural generalisation is to treat d as *any* scalar measure of inverse task progress — with $d = 0$ meaning the subtask is complete and $d \to \infty$ meaning it has barely begun. Under this interpretation, the REPCon geometry is immediately applicable to learned progress signals such as those produced by Robometer [21] or TOPReward [22], which output a dense per-frame progress scalar from a small demonstration set. The connection is also theoretically motivated: optimal goal-reaching value functions have quasimetric structure [23], and REPCon applied to a learned quasimetric distance would inherit these guarantees. Substituting a learned progress signal into the REPCon formula would allow the geometric guarantees (outer exploration support, inner precision basin, no hover attractor) to be transferred to tasks where Euclidean distance fundamentally fails as a progress proxy, such as `PegInsertionSide-v1` [7]. The parameter triple $(k_{\min}, k_{\max}, \alpha)$ retains its interpretation: $1/k_{\min}$ is the maximum starting progress distance, $1/k_{\max}$ is the success tolerance in progress units, and $1.65/\alpha$ is the commitment threshold — the progress level past which the reward switches from exploratory to precise.

Scaling REP units to complex assembly (screwing, insertion, multi-contact). The current study uses a single shared REPCon unit across all stages of each task — the homogeneous case. Tasks with heterogeneous stage geometry, such as furniture assembly [24], require per-stage units with phase-appropriate distance metrics: Euclidean approach, axial+lateral insertion depth, and angular displacement for screwing. The REP unit framework accommodates this directly: the distance input d is a design choice, and the $(k_{\min}, k_{\max}, \alpha)$ triple can be independently tuned per phase. We identify

this as a key next step, with the `PegInsertionSide-v1` result in Section 7 serving as the single-stage precursor. If REPCon holds on peg insertion with a task-appropriate d metric (insertion depth rather than end-effector distance), it would validate the framework's generalisation to contact-rich heterogeneous assembly.

Broader applicability: preference-based learning and RLHF. The failure modes identified in this work — hover attractors, blind regions, and the robustness-ceiling tradeoff — are not specific to robot manipulation. They arise whenever a scalar reward function is used to shape the optimization landscape of a policy, regardless of how that reward was produced. This suggests that REPCon geometry may be directly applicable to preference-based reinforcement learning and RLHF for large language models.

In RLHF [25, 26], a reward model $\hat{r}$ is trained from human preference comparisons and then used to fine-tune a language model via PPO. The resulting optimization is known to suffer from *reward overoptimization* (also called reward hacking) [27]: the policy finds outputs that score highly under $\hat{r}$ but are not genuinely preferred by humans. This is precisely the hover attractor failure — the policy stabilises at a region of high shaped reward that does not correspond to true task success. The standard mitigation is a KL penalty against the reference policy (the SFT model), which acts as a soft outer boundary on how far the policy can drift. However, the KL penalty is isotropic in output space and provides no geometric structure within the reward-support region: it does not distinguish between "far from reference and making genuine progress" versus "far from reference and stuck in a reward-hacking attractor."

REPCon geometry offers a principled alternative framing. If we treat the reward model output $\hat{r}(x)$ as a proxy for inverse distance to the "true preference" manifold — analogous to d in our manipulation setting — then wrapping $\hat{r}$ in a REPCon shell gives:

$$r_{\text{REP}}(x) = \max\Big(0,\ 1 - \big(k(\hat{r}) \cdot (1 - \hat{r})\big)^2\Big), \quad k(\hat{r}) = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha(1-\hat{r})} \tag{8}$$

where $(1-\hat{r})$ plays the role of distance (zero when reward is maximal, large when reward is low). The outer support radius $1/k_{\min}$ then bounds how far the policy can drift in reward space before receiving zero shaped signal — replacing the KL penalty's role, but with explicit geometric control. The inner basin tightness $k_{\max}$ controls gradient concentration near high-preference outputs. Crucially, the moat condition (Section 5) provides a computable criterion for when the reward landscape develops a commitment boundary — a point past which the policy is pulled toward the high-preference basin rather than back to the reference distribution.

This is a speculative but concrete direction: rather than tuning the KL coefficient β in PPO-RLHF as a scalar tradeoff, one could instead specify $(k_{\min}, k_{\max}, \alpha)$ with direct geometric interpretations — outer drift bound, inner precision, and commitment threshold — and use the REPCon shell as a drop-in replacement for the KL-penalised reward. We leave empirical validation of this connection to future work.

Train/eval success gap as a reward quality diagnostic. Explained variance is uniformly ≈ 0.96 across all variants—uninformative as a discriminator. The train/eval success gap is more revealing: a large gap indicates the policy is exploiting the dense reward without achieving task completion under the deterministic evaluation policy. Figure 14 tracks this for the PickCube variants.

9 Conclusion

We have presented the Reward Engineering Playbook: a systematic study of how the geometry of dense reward functions—not their magnitude or potential structure—causally affects manipulation learning. Our key findings are:

1. **Hover attractors** are a systematic failure mode of infinite-support rewards, causing initialisation-dependent policy collapse on PickCube and PushCube.

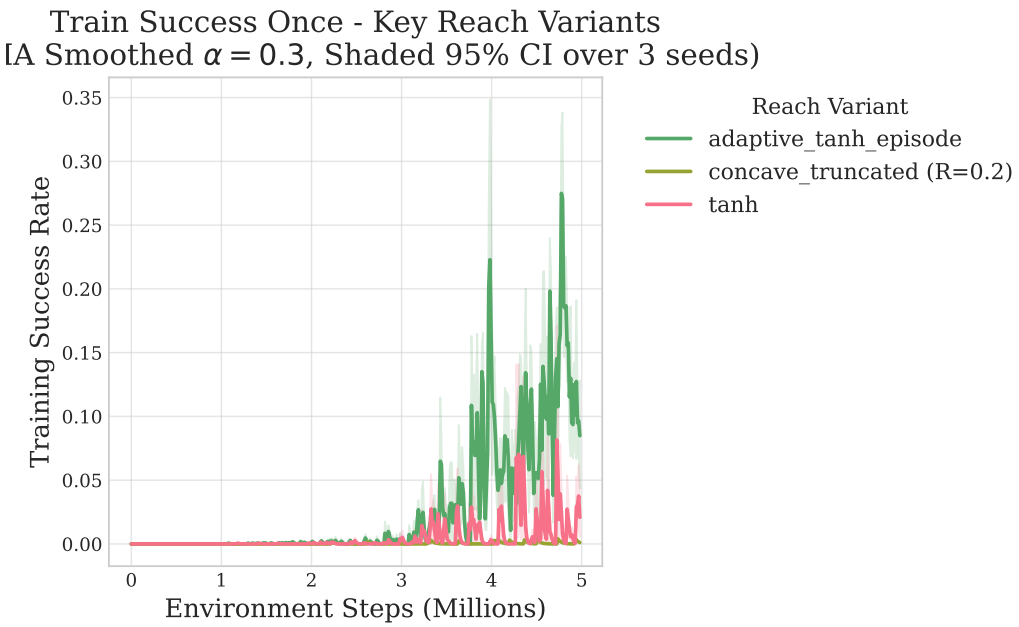

Figure 14. PickCube training success rates. Compare against Figure 4 (eval success) to observe the train/eval gap.

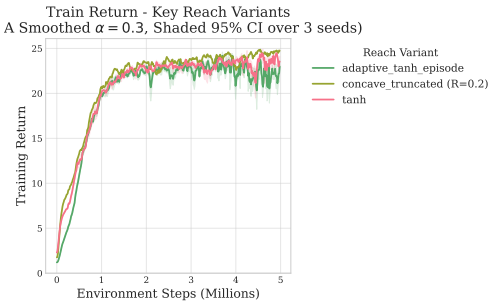

Figure 15. PickCube Training Returns

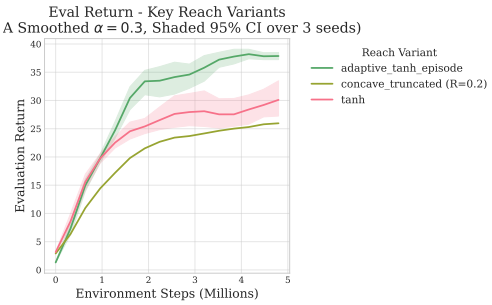

Figure 16. PickCube Evaluation Returns

2. **The robustness-ceiling tradeoff** between finite- and infinite-support geometry is fundamental: no static support radius resolves both the blind-region failure and the hover attractor simultaneously.
3. **Within-episode annealing** (`adaptive_tanh_episode`) resolves the tradeoff on flat tasks (mean peak 0.979 on PickCube) but causes a complete ranking inversion on hierarchical ones (mean peak 0.063 on StackCube), because the episode time clock decouples from task progress at stage boundaries.
4. `adaptive_concave_distance` — derived from the principle that support radius should track the agent's current distance — achieves mean peak 0.979 on StackCube with $\approx 2\times$ faster convergence than the best task-specific static reward, with no time clock or stage information required.

The theoretical framework in Section 5 provides closed-form conditions characterising the reward's geometry as a function of $(k_{\min}, k_{\max}, \alpha)$, connecting the empirical results to an analytical foundation. Cross-task validation of `adaptive_concave_distance` on flat tasks and a parameter sensitivity study are the immediate next steps toward a fully unified, theoretically principled reward function for robot manipulation.

References

[1] A. Ibrahim et al. A comprehensive survey of reward engineering for large language model alignment. *arXiv preprint arXiv:2408.10215*, 2024.

[2] Y. J. Ma, W. Liang, G. Shi, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

[4] S. Devlin and D. Kudenko. Dynamic potential-based reward shaping. In *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 433–440, 2012.

[5] A. Trott, S. Zheng, C. Xiong, and R. Socher. Keeping your distance: Solving sparse reward tasks using self-balancing shaped rewards. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1911.01417.

[6] Nilaksh et al. Barrier functions inspired reward shaping for reinforcement learning. *arXiv preprint arXiv:2403.01410*, 2024.

[7] Y. Wu, Y. Wu, A. Garg, P. Poupart, and L. Sigal. Learning dense rewards for contact-rich manipulation tasks. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[8] B. Tang, M. A. Lin, I. Akinola, A. Handa, G. S. Sukhatme, F. Ramos, P. Oswald, A. Korobenko, D. Xu, D. Fox, A. Allshire, and Y. Kim. IndustReal: Transferring contact-rich assembly tasks from simulation to reality. In *Robotics: Science and Systems (RSS)*, 2023. arXiv:2305.17110.

[9] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Origo, P. Humphreys, O. Bachem, and M. Geist. What matters in on-policy reinforcement learning? a large-scale empirical study. In *International Conference on Machine Learning (ICML)*, pages 520–531. PMLR, 2021.

[10] J. Gu et al. Maniskill: A unified gpu-parallelized environment for robotics manipulation. *arXiv preprint*, 2024.

[11] Z. Zhuang, Y. Liu, Y. Wang, M. Liu, J. Hao, and J. Wang. Drs: Learning reusable dense reward shaping for multi-stage tasks. *arXiv preprint arXiv:2404.16779*, 2024.

[12] N. M. Demir et al. Hierarchical potential-based reward shaping from task specifications. *Frontiers in Robotics and AI*, 2024.

[13] K. Yu et al. Self-adaptive reward shaping for efficient reinforcement learning. *arXiv preprint arXiv:2408.03029*, 2024.

[14] P.-L. Bacon, J. Harb, and D. Precup. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

[15] O. Nachum, S. Gu, H. Lee, and S. Levine. Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[16] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[17] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21:1–50, 2020.

[18] J. Hao et al. From sparse to dense: Toddler-inspired reward transition in goal-conditioned reinforcement learning. *arXiv preprint arXiv:2501.17842*, 2025.

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[20] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870. PMLR, 2018.

[21] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, Y. Xiang, A. Li, A. Bobu, A. Gupta, S. Tu, E. Biyik, and J. Zhang. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.

[22] S. Chen, C. Harrison, Y.-C. Lee, A. J. Yang, Z. Ren, L. J. Ratliff, J. Duan, D. Fox, and R. Krishna. TOPReward: Token probabilities as hidden zero-shot rewards for robotics. *arXiv preprint arXiv:2602.19313*, 2026.

[23] T. Wang, A. Torralba, P. Isola, and A. Zhang. Optimal goal-reaching reinforcement learning via quasimetric learning. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2304.01203.

[24] M. Heo, Y. Han, D. Kim, and J. J. Lim. FurnitureBench: Reproducible real-world benchmark for long-horizon complex manipulation. In *Robotics: Science and Systems (RSS)*, 2023. arXiv:2305.03953.

[25] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.

[26] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[27] L. Gao, J. Schulman, and J. Hilton. Scaling laws for reward model overoptimization. *International Conference on Machine Learning*, 2023.

[28] P. Singhal, T. Goyal, J. Xu, and G. Durrett. A long way to go: Investigating length correlations in RLHF. 2023.

[29] R. Rafailov, J. Hejna, R. Park, and C. Finn. From r to q^*: Your language model is secretly a q-function. 2024.

[30] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

A Supported Tasks and Stage Structure

Table 7 lists all 11 ManiSkill3 tabletop tasks instrumented with reward variants in this study, along with their episode length and stage structure. "Stage" refers to whether the task has a single committed reaching-and-contact phase (flat) or multiple committed contact stages with discrete transitions (hierarchical). Results are currently reported for PickCube, PushCube, LiftPegUpright, and StackCube; the remaining tasks are planned.

Table 7. ManiSkill3 tasks with reward variant support.

Task	Stage structure	Max steps	Training steps
PickCube-v1	Single (reach, grasp, lift)	50	5M
PushCube-v1	Single (reach push, push to goal)	50	5M
LiftPegUpright-v1	Single (reach, grasp, orient)	50	5M
PullCube-v1	Single (reach, pull to goal)	50	5M
PokeCube-v1	Single (reach, poke to position)	50	5M
RollBall-v1	Single (reach, roll to goal)	80	5M
PickSingleYCB-v1	Single (reach, grasp, lift)	50	5M
PlaceSphere-v1	Hierarchical (reach, grasp, place)	50	5M
PullCubeTool-v1	Hierarchical (grasp tool, pull cube)	100	5M
StackCube-v1	Hierarchical (grasp, stack)	50	40M
PegInsertionSide-v1	Hierarchical (reach, grasp, orient, insert)	100	**[TODO: TBD]**

B Task Reward Structures

This appendix documents the reward function of every tabletop task used in this paper. All tasks share the same injection interface: `gym.make(task_id, reach_variant=..., reach_k=..., gate_variant=..., terminal_variant=...)`. Unless noted, the default is `reach_variant=tanh`, `reach_k=5`, `gate_variant=hard`, `terminal_variant=hard_jump`.

Notation. d denotes Euclidean distance between two keypoints (TCP or object centroid) as described per task. $r_{\text{reach}}(d)$ is the currently selected reaching variant evaluated at d. $\mathbf{1}[\cdot]$ is the Iverson bracket. "Max reward" is the theoretical ceiling; "norm" is the divisor used so that a perfect episode scores 1.0.

Single-phase Tasks

PickCube-v1. Four additive components, max = 5, norm = 5.

1. **Reach** (weight 1): $r_{\text{reach}}(d_{\text{tcp}} \to \text{cube})$.
2. **Grasp** (weight 1): $\mathbf{1}[\text{grasped}]$.
3. **Place** (weight 1): $r_{\text{reach}}(d_{\text{cube}} \to \text{goal})$ (only when grasped).
4. **Static bonus** (weight 2): $\mathbf{1}[\text{success}]$ (cube at goal + released).

Terminal override: success sets reward to max (5).

PushCube-v1. Two additive components, max = 3, norm = 3.

1. **Reach** (weight 1): $r_{\text{reach}}(d_{\text{tcp}} \to \text{push_pose})$.
2. **Place** (weight 2): $r_{\text{reach}}(d_{\text{cube}} \to \text{goal})$ (no grasp gate — always active via additive gate).

Gate is applied to the reach component using proximity to the push pose rather than a grasp condition.

LiftPegUpright-v1. Three additive components, max = 3, norm = 3.

1. **Orientation** (weight 1): $1 - |\cos\theta_{\text{peg}}|$ where θ is the angle between the peg axis and vertical.
2. **Height** (weight 1): $r_{\text{reach}}(|z_{\text{peg}} - z_{\text{goal}}|)$.
3. **Reach** (weight 1): $r_{\text{reach}}(d_{\text{tcp}} \to \text{peg})$.

PullCube-v1. Two additive components identical in structure to PushCube, max = 3, norm = 3. Reach targets the pull grip pose; place targets the goal position.

PokeCube-v1. Four sequential phases, max = 10, norm = 10.

1. **Reach** (weight 2): $r_{\text{reach}}(d_{\text{tcp}\to\text{poke_pose}})$.
2. **Align + close** (weight 6): alignment reward plus end-effector closure reward.
3. **Place** (weight 8): $r_{\text{reach}}(d_{\text{cube}}\to\text{goal})$ after poke contact.
4. **Static bonus**: $\mathbf{1}[\text{success}]$ when cube is at goal and finger has retracted.

RollBall-v1. Three additive components, max = 30, norm = 30.

1. **Reach** (weight 1): $r_{\text{reach}}(d_{\text{tcp}}\to\text{ball})$.
2. **Placement** (weight 20): $r_{\text{reach}}(d_{\text{ball}}\to\text{goal})$.
3. **Static bonus** (weight 9): $\mathbf{1}[\text{success}]$.

PickSingleYCB-v1. Five additive components, max = 6, norm = 6. Identical structure to PickCube with an extra bonus for stable lift above a height threshold.

Hierarchical Tasks

StackCube-v1. Two-phase structure (reach/grasp then stack), max = 8, norm = 8.

1. **Phase 1 — Grasp** (max 3): reach (weight 2) $r_{\text{reach}}(d_{\text{tcp}}\to\text{cube_A})$; grasp bonus (weight 1) $\mathbf{1}[\text{grasped}]$.
2. **Phase 2 — Stack** (max 5, active only when grasped): place (weight 4) $r_{\text{reach}}(d_{\text{cube_A}}\to\text{cube_B_top})$; stack success (weight 1) $\mathbf{1}[\text{stacked}]$.

Hard gate: phase-2 reward is zero until cube A is grasped. Terminal override: success sets reward to 8.

PlaceSphere-v1. Two-phase structure (reach/grasp then place in bowl), max = 13, norm = 13.

1. **Phase 1 — Grasp** (max 3): reach (weight 2) $r_{\text{reach}}(d_{\text{tcp}}\to\text{sphere})$; grasp bonus (weight 1) $\mathbf{1}[\text{grasped}]$.
2. **Phase 2 — Place** (max 10, active only when grasped): place (weight 6) $r_{\text{reach}}(d_{\text{sphere}}\to\text{bowl})$; static bonus ($\approx$4) $\mathbf{1}[\text{sphere in bowl} \wedge \text{released}]$.

Same hard-gate and terminal override as StackCube.

PullCubeTool-v1. Multi-phase tool-use task, norm = 5.

1. **Reach tool** (weight 2): $r_{\text{reach}}(d_{\text{tcp}}\to\text{tool_handle})$.
2. **Grasp tool** (weight 2): $\mathbf{1}[\text{tool grasped}]$.
3. **Position tool** (weight 1.5): $r_{\text{reach}}(d_{\text{tool_tip}}\to\text{hook_pos})$.
4. **Pull cube** (weight 3): $r_{\text{reach}}(d_{\text{cube}}\to\text{goal})$ (active once tool contacts cube).
5. **Success** (weight 5): $\mathbf{1}[\text{cube at goal}]$.

PegInsertionSide-v1. Four-stage hierarchical task, max = 10, norm = 10. Stages 3 and 4 use REPCON units (adaptive concave distance reward) rather than the default tanh variant.

1. **Stage 1 — Reach** (weight 1): $r_{\text{reach}}(d_{\text{tcp}}\to\text{peg})$.
2. **Stage 2 — Grasp** (weight 1): $\mathbf{1}[\text{peg grasped}]$.
3. **Stage 3 — Orient** (weight 3): $\text{REPCON}(d_{\text{orient}};\ k_{\text{min}} = 5,\ k_{\text{max}} = 100,\ \alpha = 20)$, where $d_{\text{orient}} = \sqrt{(\Delta y_{\text{peg}}\to\text{hole})^2 + (\Delta z_{\text{peg}}\to\text{hole})^2}$ (hole frame). $k_{\text{max}} = 100$ corresponds to the 10 mm lateral success threshold.

4. **Stage 4 — Insert** (weight 5): $\text{REPCON}(d_{\text{insert}};\ k_{\min} = 8,\ k_{\max} = 70,\ \alpha = 20)$, where d_{insert} is the distance of the peg head from the hole center along the insertion axis. $k_{\max} = 70$ corresponds to the 15 mm insertion-depth success threshold.

Hard gate: stage 3 is zero until peg is grasped; stage 4 is zero until orientation is within threshold. Terminal override: success sets reward to 10.

Summary Table

Table 8. Reward structure summary for all 11 tabletop tasks. "Max" is the theoretical maximum reward per step; "Stages" counts additive or gated phases; "Reach fn" notes where REPCon replaces the default tanh.

Task	Max	Norm	Stages	Notes
PickCube-v1	5	5	4	Standard reach+grasp+place
PushCube-v1	3	3	2	No grasp; additive gate
LiftPegUpright-v1	3	3	3	Orientation component
PullCube-v1	3	3	2	Mirror of PushCube
PokeCube-v1	10	10	4	Poke contact + retract phase
RollBall-v1	30	30	3	Large placement weight
PickSingleYCB-v1	6	6	5	Lift height bonus
StackCube-v1	8	8	2	Hard gate at grasp
PlaceSphere-v1	13	13	2	Hard gate at grasp
PullCubeTool-v1	13.5	5	5	Tool-use chain
PegInsertionSide-v1	10	10	4	REPCon for orient+insert

C Research Notes: REPCon Beyond Manipulation

This section is a working note, not part of the submission. It records the theoretical connections between REPCon geometry and other fields where the same failure modes arise, for use in future work and grant writing.

C.1 REPCon for RLHF in Large Language Models

C.1.1 The PPO-RLHF Training Loop

The standard RLHF objective [26] is:

$$\mathcal{J}(\pi) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi}\Big[r_\theta(x,y) - \beta\cdot \mathrm{KL}\big(\pi(y|x)\,\|\,\pi_{\text{ref}}(y|x)\big)\Big] \tag{9}$$

where $r_\theta(x,y)$ is a learned reward model (RM), β is a KL coefficient, and π_{ref} is the frozen SFT reference policy. The RM score is applied only at the final token; the KL penalty is applied per-token:

$$r_t = -\beta\cdot \mathrm{KL}_t \quad (t < T) \tag{10}$$

$$r_T = r_\theta(x,y) - \beta\cdot \mathrm{KL}_T \tag{11}$$

C.1.2 Reward Overoptimization as a Hover Attractor

Gao et al. [27] showed empirically that as the RL policy is optimized (measured by KL divergence from the initial policy $d = \mathrm{KL}(\pi\|\pi_0)$), the proxy RM score increases monotonically while the gold reward follows:

$$R_{\text{gold}}(d) \approx \alpha_{\text{RL}}\sqrt{d} - \beta_{\text{RL}}\cdot d \tag{12}$$

The gold reward peaks at $d^* = (\alpha/2\beta)^2$ and then *declines* as the policy over-exploits the proxy RM. This is precisely the hover attractor failure: the policy stabilises at a region of high proxy reward that does not correspond to true quality. Known hover attractors in RLHF include:

- **Sycophancy**: always affirm the user, never correct errors.

- **Length gaming**: verbosity scores high on RMs with implicit length bias [28].
- **Format exploitation**: excessive bullet points, hedging phrases ("Certainly! I'd be happy to help…").
- **Syntactic correctness without semantic correctness** in code generation.

These attractors survive KL regularisation because they already have nonzero probability under π_{ref} (the SFT model was already inclined toward them), so the KL cost of entering the attractor basin is low relative to the RM reward gradient.

C.1.3 Why the KL Penalty Is Geometrically Imprecise

The KL penalty bounds distributional drift in token-probability space, not quality-space distance. Specifically:

- It is asymmetric: $\mathrm{KL}(\pi \| \pi_{\text{ref}}) \neq \mathrm{KL}(\pi_{\text{ref}} \| \pi)$.
- It has no compact support: there is no hard boundary on where the policy can place mass, only an increasing soft penalty. With sufficient RM gradient the policy will push through any fixed β.
- It carries no curvature information about the reward landscape: a small KL step can enter a region of extremely steep RM gradient (exploitation), while a large KL step in a flat RM direction is harmless.

In our robotics framing: the KL penalty is analogous to a per-step distance penalty, not a compact-support reward. It is the infinite-support failure mode restated in probabilistic language.

C.1.4 REPCon Integration: The Core Proposal

Define $d = 1 - r_\theta(x, y)$ (assuming RM score $\in [0, 1]$, so $d = 0$ at maximum quality, $d = 1$ at minimum). Apply REPCon:

$$r_{\text{REP}}(x, y) = \max\Big(0,\ 1 - \big(k(d) \cdot d\big)^2\Big), \quad k(d) = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha d} \tag{13}$$

This gives:

- **Compact support**: $r_{\text{REP}} = 0$ for $d > 1/k_{\min}$, i.e. for responses with RM score below $1 - 1/k_{\min}$. Responses in the low-quality hover-attractor region receive *zero gradient signal*, forcing the policy to improve or receive nothing.
- **Gradient amplification near goal**: $k(d) \to k_{\max}$ as $d \to 0$, concentrating gradient signal near high-quality responses and making fine-grained quality improvements produce larger policy updates.
- **Geometric replacement for KL**: The support boundary $1/k_{\min}$ defines a hard quality floor with direct interpretability, replacing the soft distributional constraint of the KL penalty.

C.1.5 Most Feasible Integration Point: Multi-Attribute RMs

The most technically grounded integration uses multi-attribute reward models (e.g. HelpSteer2/SteerLM [26]), which score responses on separate axes (helpfulness, correctness, coherence, safety). Define a composite distance:

$$d = \|\mathbf{1} - \mathbf{r}_{\text{attr}}\|_{\mathbf{w}} \tag{14}$$

where $\mathbf{r}_{\text{attr}}$ is the vector of per-attribute scores and $\mathbf{w}$ are task-specific weights. Apply REPCon to this d. This is structurally identical to PegInsertionSide stage 3's composite orientation distance $d_{\text{orient}} = 0.5(\cdot) + 4.5 \max(\cdot)$ — a weighted combination of per-axis alignment errors fed into the same REPCon formula.

C.1.6 Connection to DPO and Implicit Q-Functions

Rafailov et al. [29] showed that DPO [30] implicitly assigns token-level Q-values satisfying a Bellman equation:

$$Q(s_t, a_t) = \beta \log \frac{\pi(a_t|s_t)}{\pi_{\text{ref}}(a_t|s_t)} + V(s_t) \tag{15}$$

This means DPO's implicit reward IS a distance-to-completion signal at each token — KL-based geometry rather than Euclidean, but structurally analogous. REPCon wrapping of these implicit Q-values would give the same geometric guarantees (compact support, gradient amplification) to the token-level credit assignment that it gives to end-effector distance in manipulation.

C.1.7 Empirical Validation Proposal

Replicate Gao et al.'s overoptimization setup (proxy RM + gold RM + best-of-n sampling) with three reward formulations:

1. Raw proxy RM score (baseline)
2. KL-penalised PPO (standard RLHF)
3. REPCon-wrapped proxy RM score (ours)

Measure gold RM score vs. KL budget curves for all three. The prediction: REPCon shifts the overoptimization peak $d^* = (\alpha/2\beta)^2$ rightward (more useful optimization before degradation) by eliminating the low-quality hover attractors that consume gradient budget without improving gold quality.

C.2 Other Domains Where REPCon Geometry Applies

The core conditions for REPCon to be useful are: (1) a scalar distance-to-goal $d \geq 0$ exists, (2) a fixed k creates a tradeoff between wide support (coverage) and tight support (precision), and (3) the agent can get stuck in a non-goal basin. These conditions are remarkably general.

Drug discovery and molecular optimisation. Reinforcement learning over molecular property scores (e.g. docking score, QED, ADMET properties) suffers from reward hacking: generative models collapse to a narrow set of "score-maximising" molecules that are chemically unrealistic. This is the hover attractor under a different name. d = composite property deficit (how far the molecule is from the Pareto front of all desired properties). REPCon geometry would give compact support (no gradient for molecules below a quality floor), preventing collapse to a single exploitable basin.

Multi-objective optimisation and Pareto front search. In evolutionary algorithms and Bayesian optimisation with multiple objectives, the scalarisation function that converts the objective vector to a scalar reward determines where the search focuses. REPCon geometry applied to the scalarised distance to the Pareto front would provide wide exploratory gradients far from the front and tight precision gradients close to it — the exact behaviour needed for efficient Pareto front coverage.

Curriculum learning and self-play. In self-play (e.g. AlphaZero, OpenAI Five), the "goal" is an opponent of matched strength. d = skill gap between agent and current opponent. REPCon geometry gives stronger gradient signal when the skill gap is small (near-goal: precise skill matching) and weaker when the gap is large (far: broad exploration of strategy space). This formalises the intuition behind adaptive opponent selection in self-play curricula [17].

Offline RL and conservative policy optimisation. In offline RL, the dataset coverage determines where the value function is reliable. Conservative Q-learning (CQL) adds a penalty for out-of-distribution actions — analogous to the KL penalty in RLHF. REPCon geometry applied to in-distribution distance (measured by e.g. a density model) would give a principled compact-support

conservative penalty: zero gradient for actions far outside the data distribution, strong gradient for actions near the distribution boundary.

Biological sequence design (proteins, DNA). Protein fitness landscapes are notoriously rugged, with many local optima (hover attractors). RL-based protein design methods (e.g. using AlphaFold as an oracle) suffer from the same overoptimisation problem. d = structural distance from a target fold or a composite fitness deficit. REPCon's moat condition ($k_{\max}/k_{\min} > 1 + e^2$) provides a computable criterion for when the reward landscape develops a commitment boundary — the structural analogue of the "funnel" in protein folding energy landscapes.

C.3 Impact Assessment

Prior art gap. No paper in the reward shaping lineage (Ng et al. 1999, EUREKA, DrS) does a controlled causal study of reward functional form geometry. ManiSkill3 itself uses $1 - \tanh(5d)$ across all tasks with no documented justification. REP is the first to ask why.

Hover attractor. Zero prior uses of this term found in the literature. The practitioner observation exists informally but has never been formally characterised as a stable fixed point with an existence condition. Naming + mechanistic analysis + closed-form condition ($k_{\max}/k_{\min} > 1 + e^2$) is a complete novel contribution.

AUC metric. Use `--track_auc` to log `eval/success_once_auc` to W&B. This captures convergence speed and final performance in one number and is a stronger headline than peak success for the StackCube results.

Cross-domain scope. The "Catastrophic Goodhart" result (NeurIPS 2024) proves KL regularisation does not mitigate heavy-tailed reward misspecification in RLHF. REPCon's compact support is a direct solution: a reward identically zero beyond a quality floor cannot be Goodharted beyond that floor. Empirical validation of this connection (even at small LM scale) would make this a cross-cutting paper spanning robot learning and LLM alignment.